# ALBERT

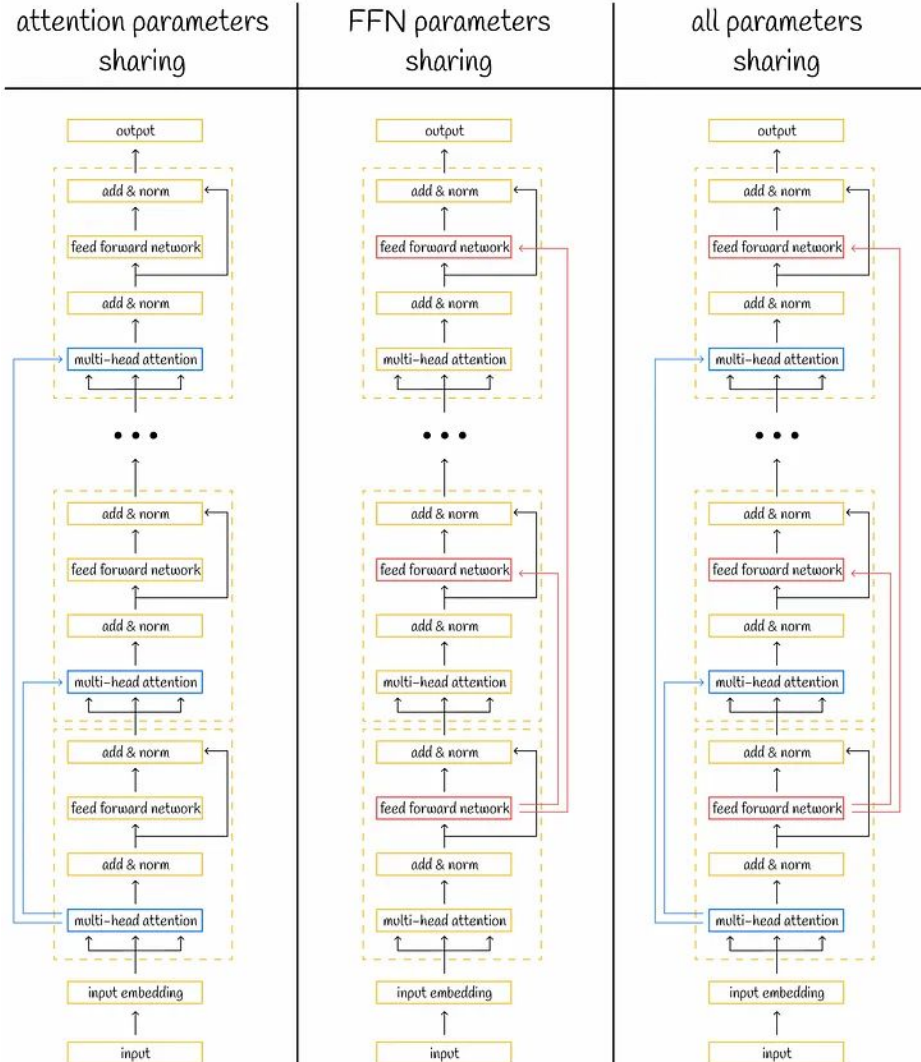
**「A Lite BERT」保持BERT的性能同時減少模型大小，主要特點包括：**

- 參數共享 (Parameter Sharing)
  - ALBERT 在所有 Transformer 層之間實現參數共享，也就是 說每一層都使用相同的參數，而不像 BERT 每層都有獨立參數 → 大幅減少了模型總參數量，降低了存儲和部署成本。
- 嵌入矩陣分解（Embedding Factorization）
  - ALBERT 將詞嵌入矩陣分解為兩個較小的矩陣：一個較小維度的詞嵌入矩陣和一個將其映射到隱藏層維度的投影矩陣。這樣做進一步壓縮了模型大小。
- 句子順序預測（SOP, Sentence Order Prediction）
  - ALBERT 捨棄了 BERT 的 Next Sentence Prediction（NSP）任務，改用 SOP 任務，要求模型判斷兩個句子的先後順序。這有助於模型更好地理解句子間的語義連貫性

# Cross-Layer (Block) Parameter Sharing

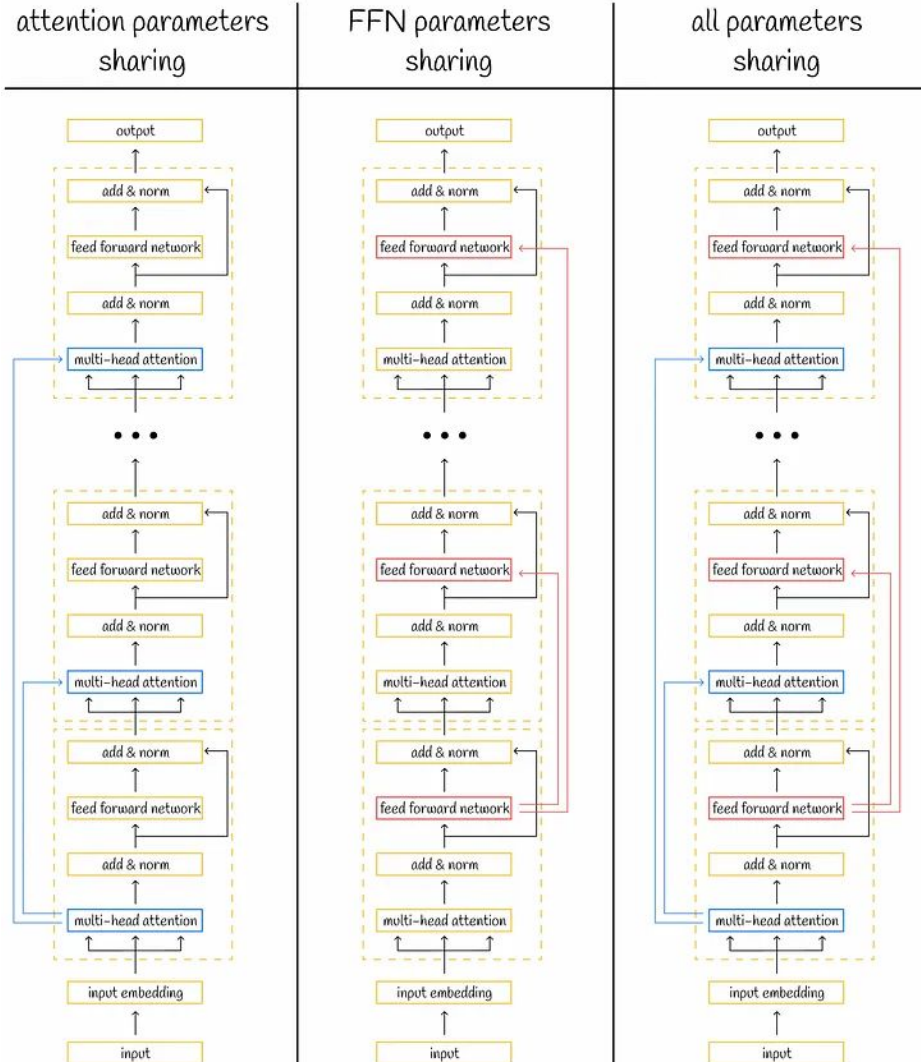
- 因共享參數，每個 encoder block 的前向傳播與後向傳播用相同的權重矩陣計算。在反向傳播時，系統會將所有隱藏層傳遞回來的梯度進行累加，最後才去更新那一組共享的權重矩陣。
- Attention parameter
  - $Q = X \cdot W_q$
  - $K = X \cdot W_k$
  - $V = X \cdot W_v$

  Shared across layers
- Default 是共享所有層的參數



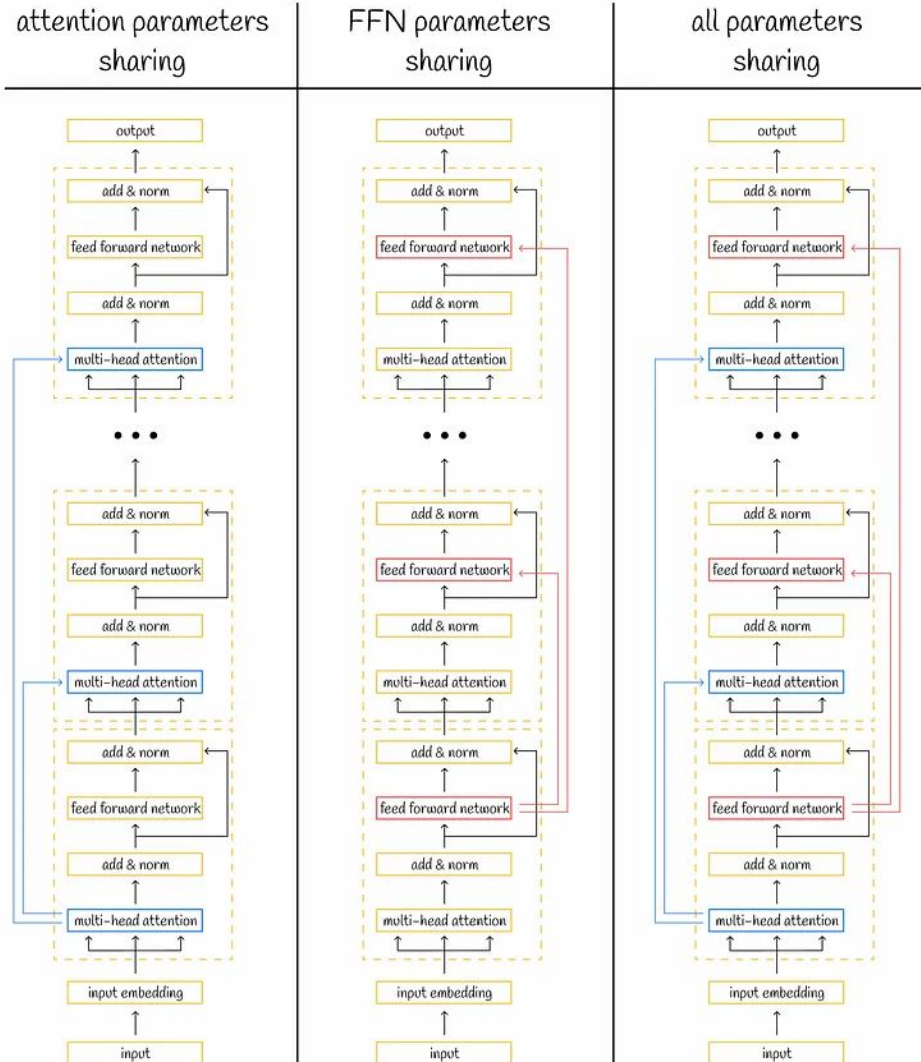
# 參數變少 ＝ 運算變快？

- Lite 指的是模型參數量(記憶體)的大幅減少，運算量(FLOPs)與推論時間並沒有減少。
- 因為資料在前向傳播時，還是得走完所有的 Encoder block 進行矩陣運算。
- 因此，ALBERT 的推論速度與相同層數的標準 BERT 基本上是一樣的。
- 雖然運算沒有變快，但因為模型佔用的 VRAM 變小，意味著在訓練模型時，可以設定更大的 Batch Size



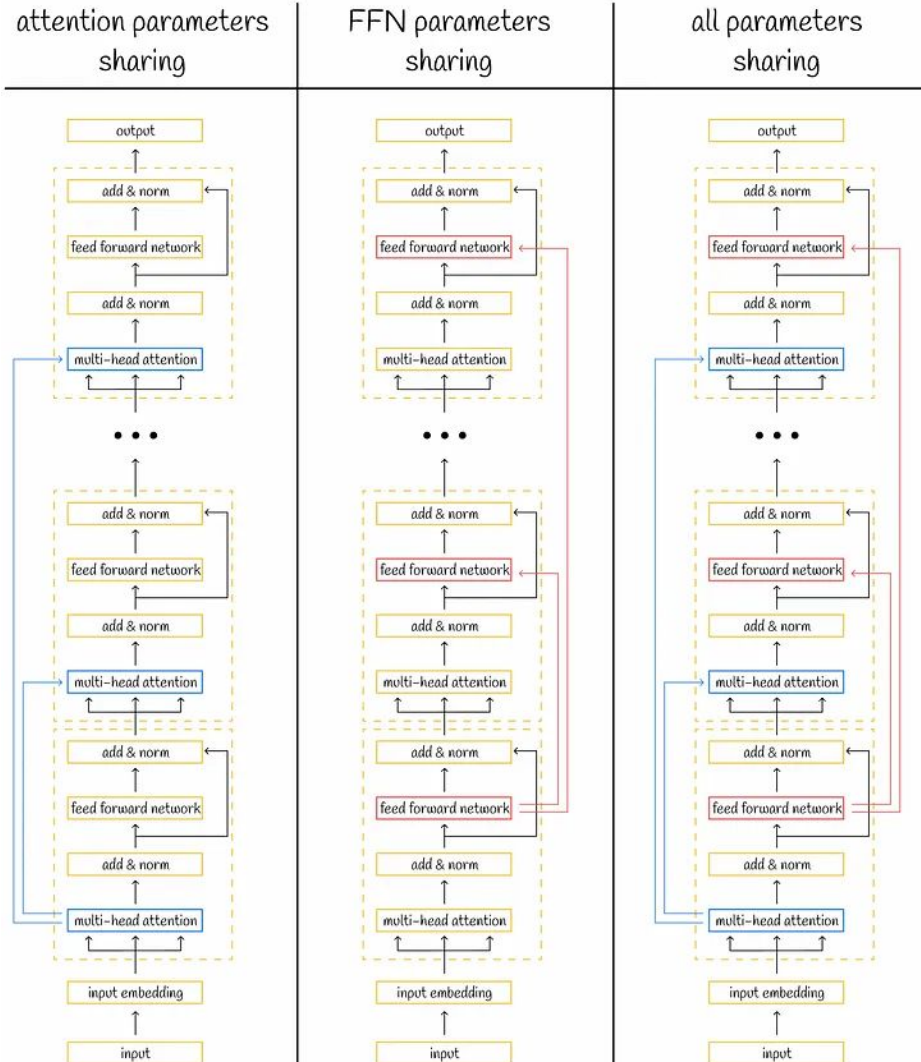
# Regularization

- Regularization is used to avoid overfitting
- 標準的BERT有12 組不同的權重可以去死記訓練資料中的細節與雜訊
- ALBERT只有一組唯一的權重，因此被迫去學習語言中最核心、最通用的特徵



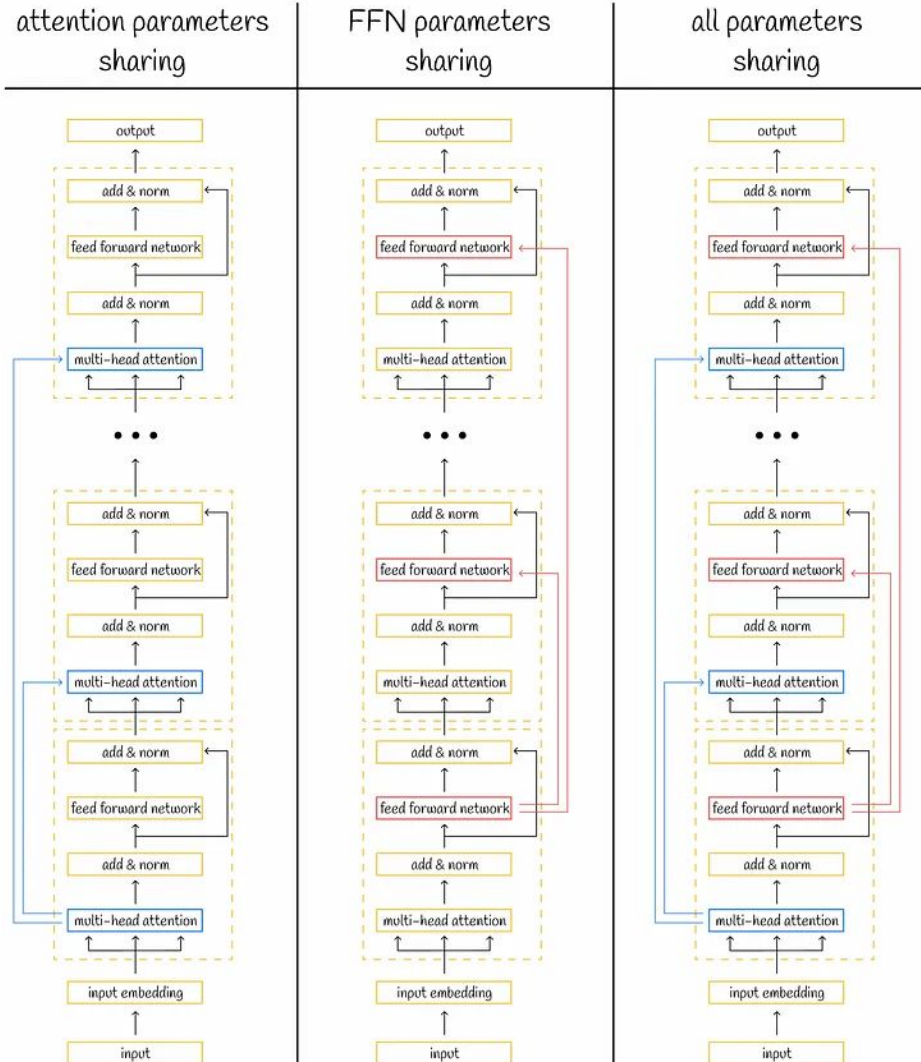
# Regularization

- ALBERT論文中提到，如果我們去追蹤一個詞彙的向量在通過每一層時的變化（e.g. 餘弦相似度的差異）會發現：
  - 在標準 BERT 中向量的變化軌跡經常是劇烈震盪的。有些層之間的轉換會突然發生巨大的改變，這表示網路的函數空間是崎嶇的。
  - 在 ALBERT 中因為每一層的轉換矩陣都一樣，向量狀態的轉移會變得非常平滑。這種層間轉換的穩定性，在數學上對應著更平滑的損失函數 landscape，能提升模型在面對未見過資料時的表現。



# Regularization

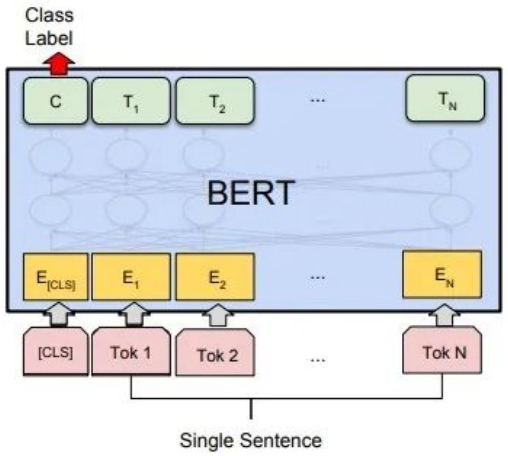
- 在非常深的非共享網路中，研究發現許多深層的 Attention Head 其實沒有在做事，它們的輸出幾乎就是輸入本身，或是只專注在沒有意義的標點符號上。
- ALBERT 則強迫同一組參數必須在每一層都 有所貢獻。因為如果這組權重在某一層不做事，它在其他所有層也都會失效，進而導致整個模型的 Loss 飆高。



# Factorized Embedding Parameterization

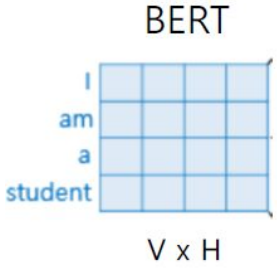
V為詞彙量(嵌入的總數)
H為嵌入維度

BERT會在嵌入層將單詞轉為固定大小的向量，對於有大量詞彙量的模型，詞嵌入層會有非常大的參數矩陣




傳統作法

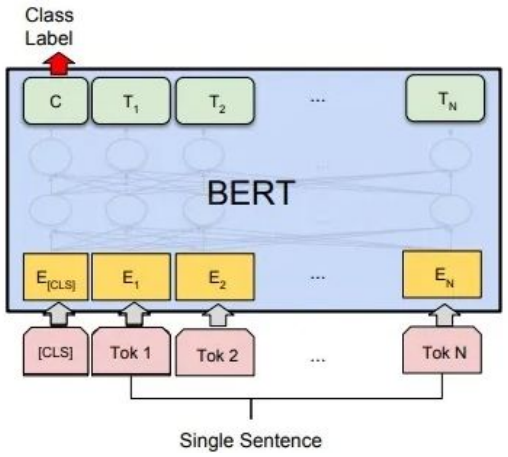
對於每個V的嵌入，需要儲存H值，產生V*H的嵌入矩陣



# Factorized Embedding Parameterization

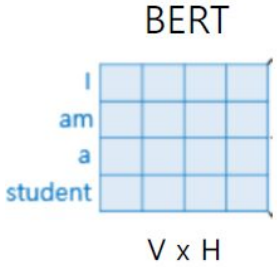
V為詞彙量(嵌入的總數)
H為嵌入維度

BERT會在嵌入層將單詞轉為固定大小的向量，對於有大量詞彙量的模型，詞嵌入層會有非常大的參數矩陣




傳統作法

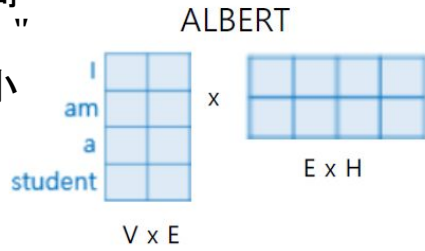
對於每個V的嵌入，需要儲存H值，產生V*H的嵌入矩陣




為了減少參數

factorized embedding parameterization

將每個詞映射到一個遠小於隱藏層的維度空間
" **引入中間嵌入層 (E)** "
透過因式分解變成較小的矩陣



## Factorized Embedding Parameterization

- 假設詞表大小 V = 30000, 隱藏層 H = 768, ALBERT 的中間層 E = 128
- 標準 BERT：30000 x 768 ≈ 2300 萬個參數
- ALBERT：(30000 x 128) + (128 x 768) ≈ 394 萬個參數

# Sentence Order Prediction

句子 A：天空突然下起大雨
句子 B：所以我把雨傘撐開

- NSP:
  正樣本：天空突然下起大雨 + 所以我把雨傘撐開
  負樣本：天空突然下起大雨 + 牛頓發明了微積分
  → 只要看有沒有下雨相關的字就好

- SOP：
  正樣本：天空突然下起大雨 + 所以我把雨傘撐開
  負樣本：所以我把雨傘撐開 + 天空突然下起大雨
  → 需要讀懂“因為...所以...”的時間順序與因果邏輯

# DistilBERT

DistilBERT = 簡化版的 BERT

- 由 Hugging Face 開發
- 是透過知識蒸餾（Knowledge Distillation）訓練而來的模型
- 保留大部分 BERT 的效能，同時減少模型大小與推論時間

「效能保留約 97%，但模型縮小 40%，速度提升約 60%」

# Why DistilBERT?

- BERT 的問題:
  - 參數多、模型大（BERT base 約 110M 參數）
  - 推論慢、記憶體需求高

- DistilBERT 的目標:
  - 在 資源有限的裝置 上也能運行（如手機、邊緣設備）
  - 提供接近 BERT 的表現，但更快速、更輕量

# DistilBERT 如何訓練？

知識蒸餾訓練流程：

- 教師模型：原始的 BERT
- 學生模型：DistilBERT
- 學生模型學習教師模型的預測，損失函數包涵：
  - 預測結果的交叉熵（Soft labels）
  - 標準的遮蔽語言建模損失（MLM Loss）
  - 隱藏層輸出的餘弦相似度（Cosine embedding loss）

DistilBERT 只保留一半層數（6 層，BERT 有 12 層）在初始化時，它是直接從教師模型中每隔一層抽取權重來初始化學生模型（例如抽取第 0, 2, 4, 6, 8, 10 層的權重），但仍保留了原本 768 維的隱藏層大小

# TinyBERT

TinyBERT = 更小、更針對特定任務優化的 BERT

- 同樣採用知識蒸餾，但額外包含：
  - 任務導向蒸餾（Task-specific distillation），如分類、問答等
  - 更細緻的中間層蒸餾（如 attention、hidden state、embedding）
- 支援不同大小版本（如 4 層、6 層），可針對應用自由調整
- TinyBERT4 vs BERT-base
  - 參數減少約87%
  - 推論速度提升約9.4倍
  - 在 GLUE 基準測試中，平均能維持 BERT-base 約 96.8% 的效能表現

# TinyBERT 的兩階段訓練策略

- 通用蒸餾（General Distillation）
  - 在預訓練階段，TinyBERT 學習 BERT 的語言知識表示能力：
  - Embedding-layer Distillation
    - 模仿 BERT 的嵌入輸出向量
  - Transformer-layer Distillation
    - 對齊 BERT 的 hidden states 與 attention weights
  - 不包含 Prediction-layer distillation（因為此階段無下游任務）

# TinyBERT 的兩階段訓練策略

- 任務蒸餾(Task-specific Distillation):
  - 在特定任務(如情感分析)上進一步微調, 提升任務表現
  - 重新蒸餾 transformer 層(optional)
  - Prediction-layer Distillation
    - 模仿 BERT 在任務上的輸出行為(機率分布)
  - 資料擴充
    - 下游任務的標註資料量通常很少。只用這些少量資料進行蒸餾會導致Student 模型難以充分學習
    - Step 1: 輸入句子 → 依機率 $p_{mask}$ 選定 Target Word
    - Step 2: 判斷 Target Word 是否被 WordPiece 切分？
      - single-piece (ex: play) → 透過 BERT MLM 預測 Top-K
      - multi-piece (ex: playing)→ 透過 GloVe Cosine Similarity 找 Top-K
    - Step 3: 從 Top-K 抽樣替換 → 輸出擴充句子

# General Distillation

- Embedding-layer Distillation
  - 使TinyBERT更小維度的embedding輸出結果更接近BERT的embedding輸出結果
  - 通常會使用一個損失函數（例如 MSE），讓學生模型（TinyBERT）的 embedding 與 BERT 的對應 embedding 盡可能相近
- Transformer-layer Distillation
  - 取k層蒸餾的方式，如：BERT的Transformer有12層，TinyBERT有4層，則取每隔3層蒸餾，TinyBERT的第1、2、3、4層transformer的輸出分別學習BERT的3、6、9、12層輸出
  - 不只是輸出特徵（hidden state），還會對 attention 分布（self-attention weights）進行對齊

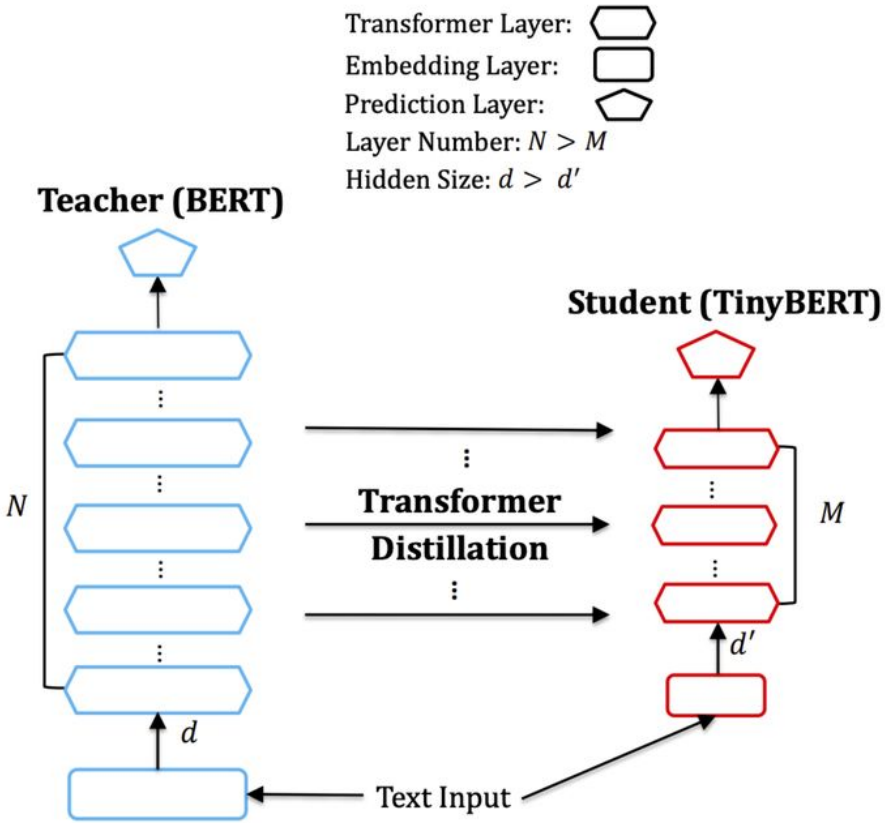
# Self-Attention Weights 對齊

- 不只是輸出結果相似，我們還希望 TinyBERT 在「看誰」這件事上也和 BERT 一樣
- 舉例來說：
  - BERT 的某一層裡，token "apple" 可能很注意 "eat"
  - 我們希望 TinyBERT 也能在這一層中把注意力分到 eat

- 所以會讓 TinyBERT 的 self-attention 分布（每個 token 對其他 token 的注意權重）去模仿 BERT 的 attention weights
- 使用 MSE 針對未經過 softmax 的原始 attention weights進行計算

# Task-specific Distillation

- 重新學習 在任務資料上, BERT 的行為
- Prediction-layer Distillation
  - 讓TinyBERT學習BERT最終輸出的行為, 在訓練過程中, 要儘量學習模仿 BERT的輸出機率分佈
  - 通常會使用 KL divergence 損失函數
- 也會再次對齊中間層(不是只學結果, 也學過程)
- 這讓 TinyBERT 不只會 general NLP, 還能針對下游任務更精準


Transformer Layer:
Embedding Layer:
Prediction Layer:
Layer Number: $N > M$
Hidden Size: $d > d'$
Teacher (BERT)
Student (TinyBERT)
$N$
$M$
Transformer
Distillation
$d$
$d'$
Text Input

# 對比：DistilBERT 的做法

- DistilBERT 只在預訓練階段做簡單的蒸餾（一次性、只對齊 prediction 和 hidden state）：

| 特徵 | DistilBERT |
| --- | --- |
| 蒸餾階段 | 僅在預訓練階段 |
| 蒸餾內容 | 最終 logits (prediction ) + 每層 hidden state |
| 是否蒸餾 attention weights | ❌ |
| 是否 task-specific | ❌ |

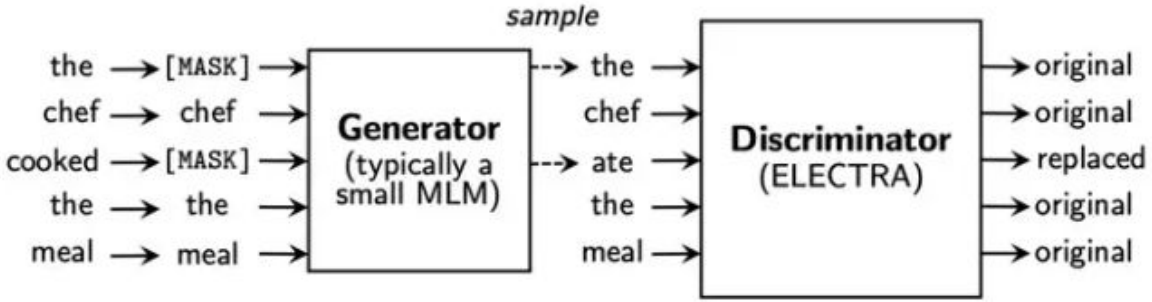
# ELECTRA

ELECTRA引入類似對抗網路(GAN)的生成器跟判別器的訓練方式

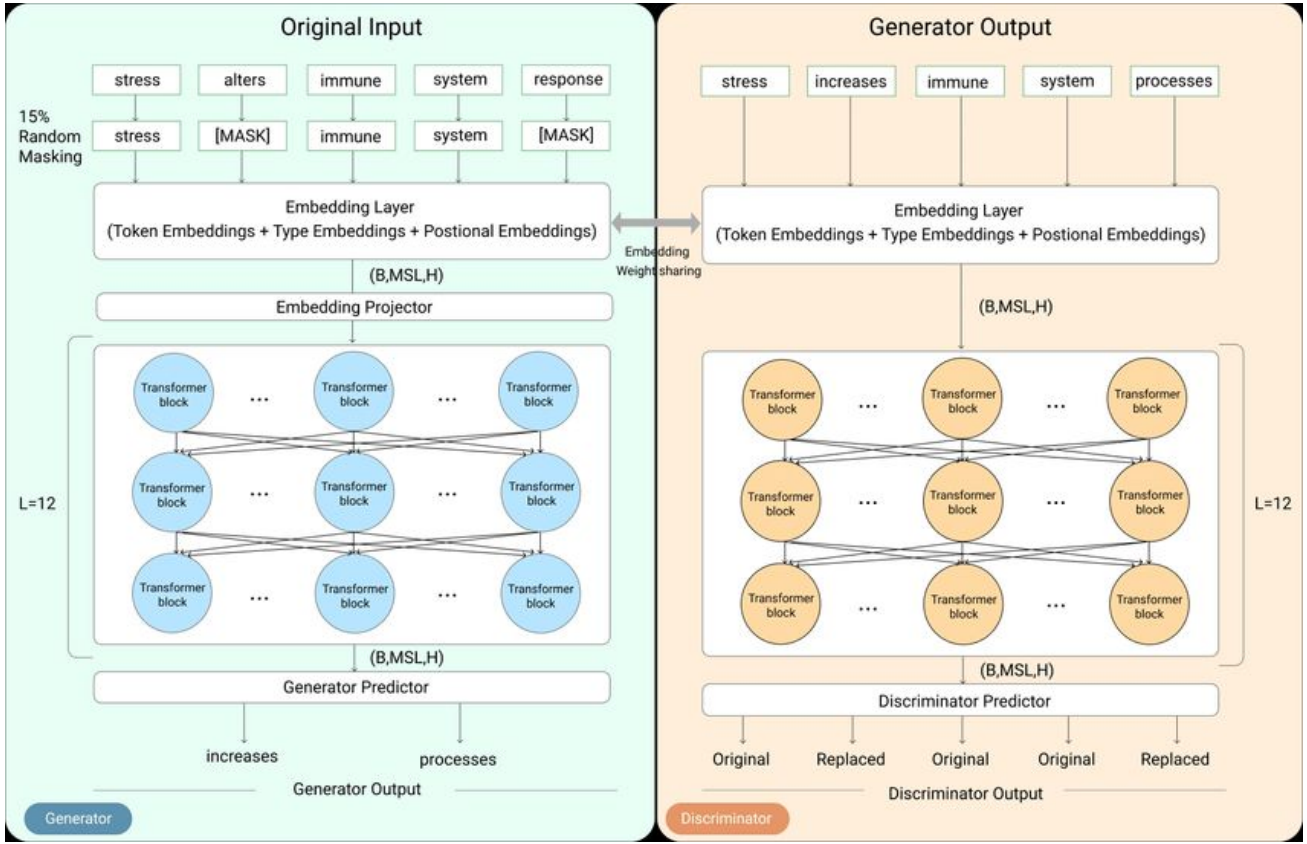
ELECTRA包含兩部分

- 生成器(小型的BERT):替換句子的某些詞
- 判別器:判斷每個單詞是否替換過

將MLM預訓練改為**替換Token檢測(Replaced Token Detection)**，**RTD**任務中，模型要區分Input的真實單詞和被替換的偽造單詞，判別器會在nput的每個Token進行預測，不僅僅是只有被mask的Token



# ELECTRA


Original Input
stress
alters
immune
system
response
15% Random Masking
stress
[MASK]
immune
system
[MASK]
Embedding Layer
(Token Embeddings + Type Embeddings + Postional Embeddings)
(B,MSL,H)
Embedding Projector
Transformer block
L=12
(B,MSL,H)
Generator Predictor
increases
processes
Generator Output
Generator
Embedding Weight sharing
Generator Output
stress
increases
immune
system
processes
Embedding Layer
(Token Embeddings + Type Embeddings + Postional Embeddings)
(B,MSL,H)
Transformer block
L=12
(B,MSL,H)
Discriminator Predictor
Original
Replaced
Original
Original
Replaced
Discriminator Output
Discriminator

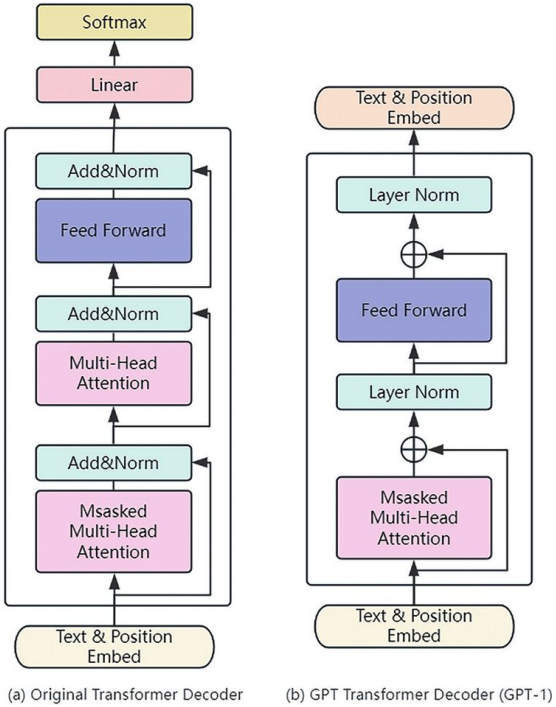
# 與 BERT 的比較

| 項目 | BERT | ELECTRA |
| --- | --- | --- |
| 預訓練任務 | MLM | RTD |
| 訓練效率 | 僅對 15% 被 mask 的 token 計算 Loss | 所有 token 皆參與 Loss 計算 |
| 使用效益 | 需較多預訓練資源 | 更快、更省資源（訓練步數較少） |
| 結果品質 | 高 | 可達或超越 BERT 相同大小的版本 |

# GPT

# 什麼是GPT？

- GPT（Generative Pre-trained Transformer）是由 OpenAI 開發的語言生成模型，基於 Transformer 的 **Decoder** 架構 所構成。
- 核心運作方式為 **自回歸（autoregressive）**語言建模：每一步僅根據前面的詞來預測下一個詞。
- 使用 **Masked Multi-Head Attention**，讓模型只能看到「當下位置以前的詞」，強迫學會序列式生成，避免偷看未來資訊。




(圖)模型結構

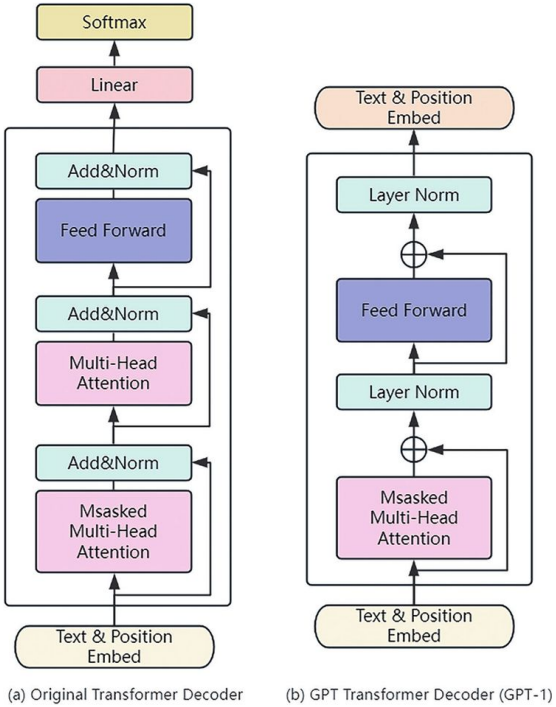
# 什麼是GPT？

Transformer decoder處理過程：

- 輸入"今天天氣如何？"，翻譯的目標"How is the weather today?"
- 開始生成輸出時，Decoder先接收開始符號<START>
- Masked Multi-Head Attention：會先關注之前的序列<START>
- Multi-Head Attention (cross-attention)：Decoder抓取輸入句子"今天天氣如何?"的相關上下文，幫助決定下一個最佳的輸出詞，例如："How"
- 重複迭代

GPT的處理：

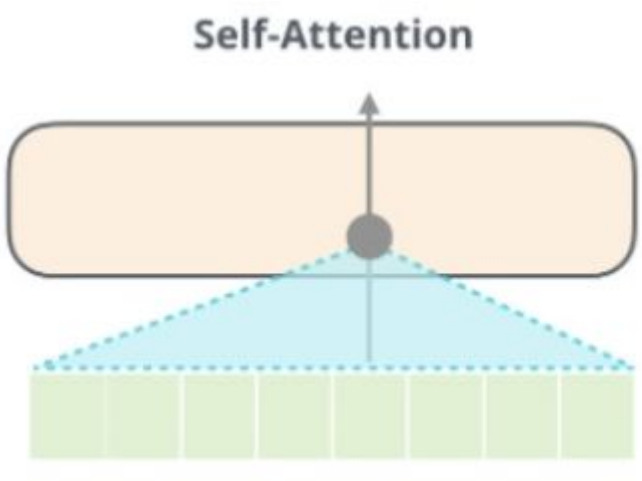
- 輸入可能是一個不完整的句子"今天天氣"
- Masked Multi-Head Attention：會先關注之前的序列"今天天氣"，每生成一個詞就考慮之前所有的詞，例如：生成"如"
- 重複迭代，最終可能生成的句子"今天天氣如何?"或"今天天氣很好"



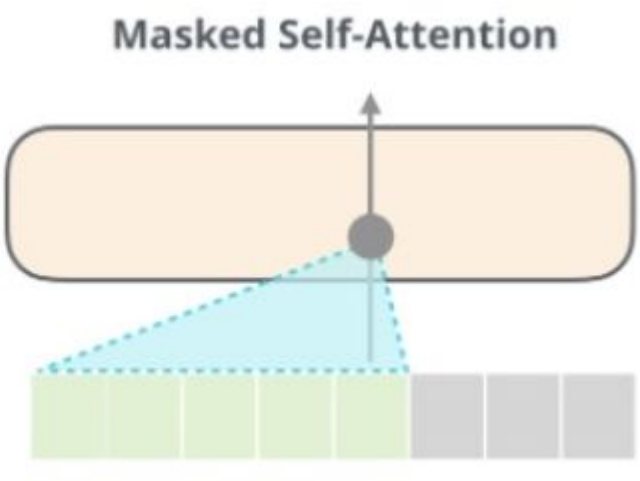

(圖)模型結構

# GPT和BERT

- BERT會將整個輸入做Self-Attention，而因為用了雙向，在**語意理解**處理上更突出
- 為了增強模型對不同任務的適應性和泛化能力，GPT採用自迴歸來避免再生成文本時利用到未來的訊息，因此再**生成任務**上較出色




BERT(Self-Attention)




GPT(Masked Self-Attention)

# Next Token Prediction

1. 經由Decoder生成出下一個有可能的詞向量
2. 通過線性層轉為Logits分數
3. 通過softmax，轉換每個詞為機率分佈

對於"今天天氣"，可能得到

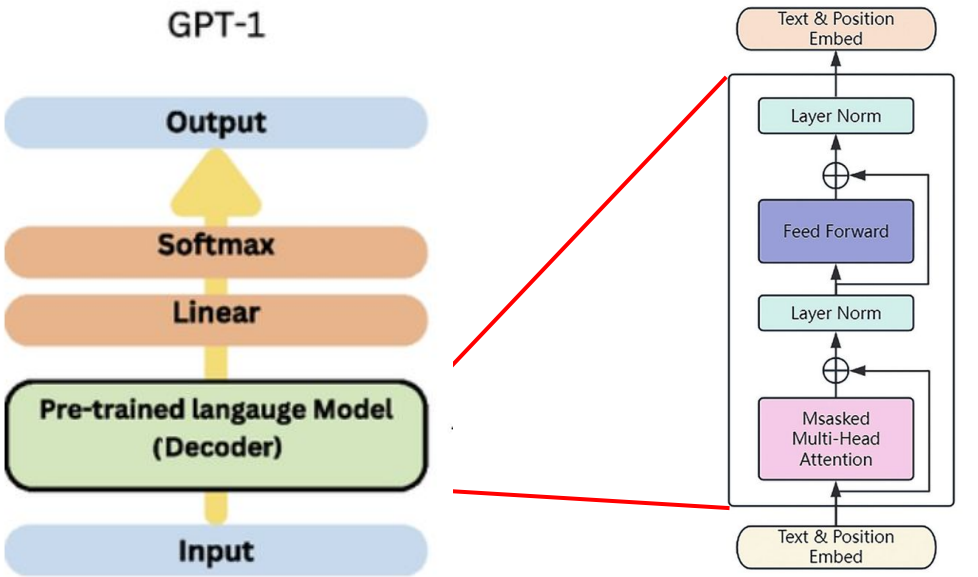
"如何"機率為0.4

"很好"機率為0.3

"很熱"機率為0.3

使用哪種採樣策略取決於設計的選擇

- Greedy Search
- Random Sampling
- Top-k Sampling
- Temperature Scaling




(b) GPT Transformer Decoder (GPT-1)

# 採樣策略比較

| 策略名稱 | 說明 | 優點 | 缺點 |
| --- | --- | --- | --- |
| Greedy Search | 每次都選機率最大的詞（Argmax） | 穩定、簡單 | 缺乏創意、可能卡住重複循環 |
| Random Sampling | 根據機率分布隨機選詞 | 多樣性高 | 容易產生不通順或不合邏輯的詞 |
| Top-k Sampling | 限制只從機率最高的前 k 個詞中選擇 | 保留隨機性又避免極端詞彙 | k 值選不好會影響品質或創意 |
| Temperature Scaling | 改變 softmax 輸出分布的「尖銳程度」; T 越低 → 越貪婪 ; T 越高 → 越隨機 | 可配合上面任何策略使用 | T 太高會胡亂生成, 太低變貪婪 |

# GPT-1

- GPT-1 的實驗驗證顯示：隨著解碼器層數的增加，模型的語言理解與生成能力會逐漸提升，證實了「大模型更聰明」的可能性。
- 儘管 GPT-1 已具備基本的 **Zero-shot** 能力（無需任務訓練即可嘗試輸出），但其效果仍不如後續版本穩定或強大。若希望在特定任務上取得更佳表現，GPT-1 通常需要進行微調。
- 微調方式：將 GPT-1 針對特定任務的標註資料進行再訓練。例如在情緒分析任務中，會使用標記有「正面／負面」情感的資料進行訓練，讓模型學會準確地預測文本的情緒傾向。

# Zero-Shot

- 模型可以處理它在訓練期間未見過的任務，可以無需針對特定任務微調，即可理解和回答問題、分類文本、翻譯語言等

  就像你讓朋友去做一個蛋糕，但他之前從來沒有做過蛋糕，也沒人專門教他們做。但因為他們知道做飯的基本知識，而嘗試根據自己的理解去完成任務

- 進行Zero-Shot時，使用者需要能夠清楚地描述他們的詢問，並且以一種結構化的方式來組織prompt，以利用預訓練的知識。

# example1：情感分析

```
Classify the following text's sentiment as positive, neutral, or negative.
Desired Format: a number, -1 for negative, 0 for neutral, and 1 for positive
Input:  [這裡放入需要分析情感的一段文字 ]
Sentiment:
```

# example2：提取關鍵字

```
Extract keywords from the below text.
Text: [這裡放入需要被篩選關鍵字的一段文字]
Keywords:
```

# example3：文章摘要

```
Generate a summary for the following text.
Text: [這裡放入長文本]
Summary:
```

# GPT-2: 打開大型語言模型時代的大門

- 發布年份：2019
- 參數數量：15 億（1.5B），是 GPT-1（1.1 億）的 10 倍以上
- 結構：48 層 Transformer Decoder（以 15 億參數版本為例）
- 訓練資料：超過 8 百萬份網頁（WebText）

# 創新與突破

- 強化 Zero-shot 能力
  - 模型不需微調也能在多種任務（翻譯、摘要、問答等）上表現不錯
- 泛化能力強
  - 能在未訓練過的任務上給出合理回答，顯示出強大的語言泛化與推理潛力
- 語言生成能力極強
  - 能生成語法正確、內容連貫、語意通順的長篇文本（但也可能出現「幻覺」）

# GPT-2

| | GPT-1 | GPT-2 |
|---|---|---|
| 模型規模 | 擁有1.1億個參數 | 參數擴展至15億 |
| 訓練數據 | 訓練數據較少(BooksCorpus) | 使用更大更廣泛的數據集<br>(WebText) |
| 性能和<br>泛化能力 | 在多種任務上都顯示很好的性能<br>但泛化能力受限於模型規模與訓<br>練數據的廣泛性 | 有了更多參數與訓練數據<br>在zero-shot任務上<br>表現更加流暢與連貫 |
| zero-shot<br>學習能力 | 對於一些任務會限制<br>zero-shot學習能力 | 即使沒有針對任務進行微調<br>也能在多種任務獲得合理結果 |
| 生成品質 | 有時不夠連貫，特別在長文本時 | 生成更連貫、更貼近人類寫的<br>長文本 |

核心思想：當模型的容量非常大且資料量足夠豐富時，僅僅靠語言模型的學習便可以完成其他有監督學習的任務，不需要在下游任務微調。

# GPT-3

- 藉由GPT-2證實規模加大的預訓練模型是可行的，因此出現了GPT-3
- 概念非常簡單粗暴，使用更多的運算資源

| | GPT-1 | GPT-2 | GPT-3 |
|---|---|---|---|
| 模型規模 | 擁有1.1億個參數 | 參數擴展至15億 | 參數擴展至1750億 |
| 訓練數據 | 訓練數據較少(BooksCorpus) | 使用更大更廣泛的數據集<br>(WebText) | 使用更大更多樣化規模的數據<br>涵蓋更廣泛的語言和上下文 |
| 性能和<br>泛化能力 | 在多種任務上都顯示很好的性能<br>但泛化能力受限於模型規模與訓練數據的廣泛性 | 有了更多參數與訓練數據<br>在zero-shot任務上<br>表現更加流暢與連貫 | 由於更大的規模，性能各種任務<br>上都有顯著提升，特別在<br>few-shot學習方面 |
| zero-shot<br>學習能力 | 對於一些任務會限制<br>zero-shot學習能力 | 即使沒有針對任務進行微調<br>也能在多種任務獲得合理結果 | 通過zero-shot及few-shot<br>在沒有額外數據情況下也能適應多種任務 |
| 生成品質 | 有時不夠連貫，特別在長文本時 | 生成更連貫、更貼近人類寫的<br>長文本 | 除了規模，還優化訓練技術，<br>以便產生準確的輸出 |

# InstructGPT

- 語言模型的大小增加不代表它能夠更好的理解使用者的意圖，事實上，語言模型可能生成不真實的訊息或毫無幫助的輸出。因此，InstructGPT的主要目標是讓語言模型更好的**遵循人類給出的指令，並實現這些指令**

- 例如：

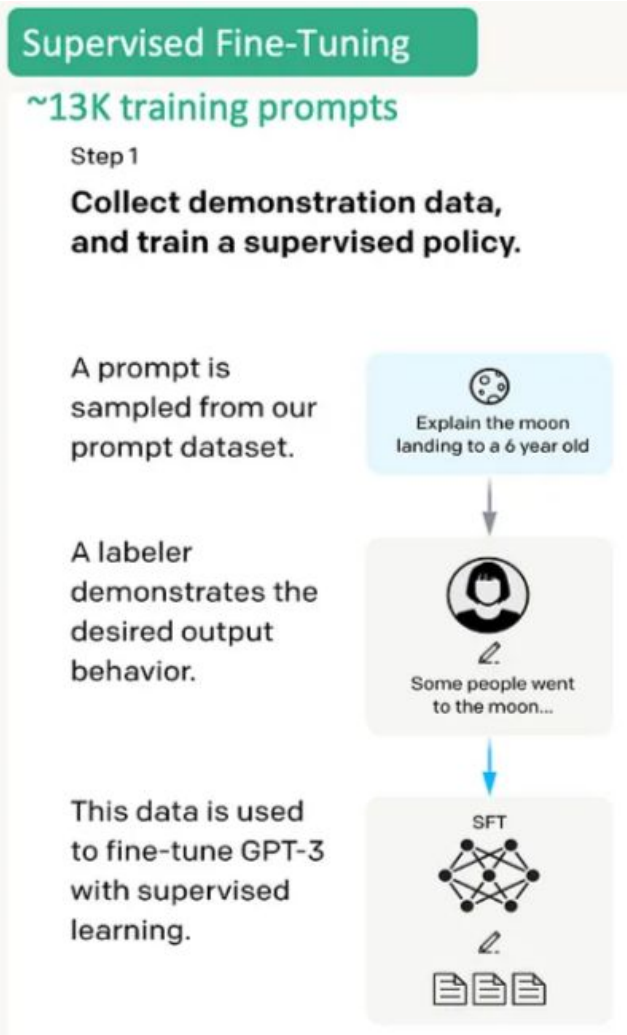
台灣最高的山是哪座？ → GPT3 → 日本最高的山是哪座？(X)
預期：玉山

# InstructGPT訓練方法

- 基於GPT-3的基礎上，使用human data進行fine-tuned

## 1.SFT：加入人工示範資料進行監督式微調

- 標註人員針對各式各樣的 Prompt，親自撰寫高品質、符合人類期望的**標準回答**
- 將這些**問題＋解答** 的配對資料加入訓練中，對 GPT-3 基礎模型進行微調
- 模型透過監督式學習，從單純的**預測下一個字**，轉變為學會**遵循指令** 並具備初步對話能力

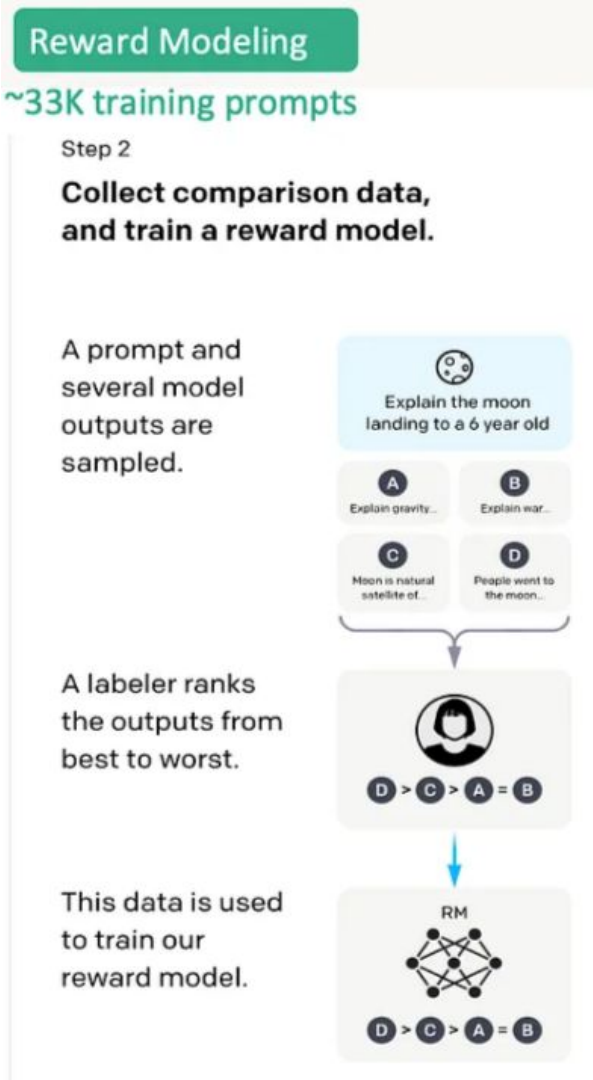


# InstructGPT訓練方法

- 基於GPT-3的基礎上，使用human data進行fine-tuned

## 2.人類老師引導文字接龍的方向：Reward Model獎勵模型

- 針對同一個問題，先讓模型產生數個不同的回答版本
- 老師會檢查這些不同的回答，並根據安全性、正確性與流暢度進行**好壞排名**
- 使用老師排名的資料，另外訓練出一個獎勵模型 (Reward Model)。它的目的是學會人類的評分標準
- 這個訓練好的 Reward Model，之後將在第三階段代替人類老師，自動且大量地為語言模型打分數



# InstructGPT訓練方法

#1

Q: 請給出五月天三首與『人生』有關的歌曲?
A:
1.《第二人生》
2.《人生海海》
3.《一半人生》

#2

Q: 請給出五月天三首與『人生』有關的歌曲?
A:
1.《志明與春嬌》
2.《人生海海》
3.《倔強》

#3

Q: 請給出五月天三首與『人生』有關的歌曲?
A:
1.《志明與春嬌》：這首歌曲講述了一個年輕人和女孩的青春故事，表達了對愛情和生活的探索和思考，是一首非常具有啟示意義的歌曲。
2.《人生海海》：這首歌曲是對人生的一種抒發和思考，歌詞告訴我們人生的旅程充滿了挑戰和變化，需要我們勇敢面對，並且我們需要珍惜每一刻。
3.《倔強》：這首歌曲表達了對於生活中的困難和挫折要堅強不屈的態度，唱出了不向命運屈服的勇氣和決心，是一首非常激勵人心的歌曲。

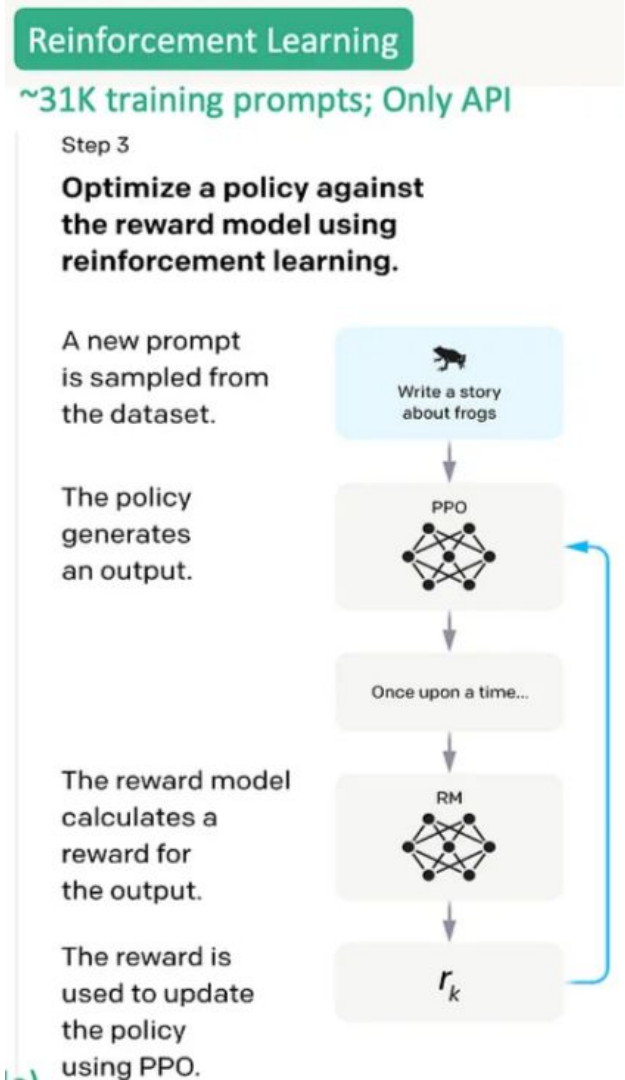
如果我們是老師，面對這些回答時給予的排序:#3 > #2 > #1，並回饋到模型中

# InstructGPT訓練方法

- 基於GPT-3的基礎上，使用human data進行fine-tuned

## 3.強化學習（PPO演算法）讓文字接龍能獲得最大獎勵

- Proximal Policy Optimization (PPO) 是一種強化學習策略。模型會不斷嘗試生成新的對話，並根據結果來調整自己的回答方式
- 型的目標是獲得最高的分數（獎勵）。它每產生一段回答，上一階段訓練好的 Reward Model 就會自動幫它打分數
- 類似於告訴模型："按照這個方法練習，你就能獲得更高分"，通過PPO，模型在多次練習中逐漸學會說什麼話，以獲得老師的正面反饋。模型就能不斷進步，變得越來越擅長與人類對話



# InstructGPT

由於人類老師只能給予教導與引導的例子有限，下面這個問題在訓練數據甚至都沒出現過

為什麼冥想之後吃襪子很重要?
**GPT3(左):**
煞有其事地跟你說，你在品嚐啟示的本質。

**InstructGPT(右):**
會說這個問題沒有明確答案。但是後面還是煞有其事地跟你說：
有些專家相信吃襪子的行為有助於大腦意識狀態的抽離
...有些其他理論提出了吃襪子的行為會給予冥想者『感官上的新體驗』。

- InstructGPT相比於GPT-3有了辨別問題的能力，能夠避開回答，但仍然有似是而非的論述
- 這是因為Reward Model的副作用，標註員往往會偏好那些看起來**結構完整、語氣自信、資訊豐富**的回答

# GPT3.5

- 基礎架構與訓練資料升級
  - 模型規模與 GPT-3 相近，但加入了大量程式碼進行混合預訓練。程式碼嚴謹的邏輯與結構，大幅跨領域提升了模型在自然語言上的 **邏輯推理** 能力。

- 強化的上下文與多輪對話能力
  - 在微調階段，資料格式變成了包含 **System、User**和**Assistant**的劇本格式
  - 讓模型學會在生成回答時，不僅看當下的問題，還能將前面的對話歷史納入考量，讓它表現出很好的連貫性
  - 能更精準理解使用者的複雜指示，並確實執行任務（如：規定字數、特定格式輸出），減少答非所問的狀況

Ex:
[System]: 你是一個專業、有禮貌的台灣高中資訊老師。你的回答要盡量通俗易懂，並且繁體中文
[User]: 老師，請問什麼是迴圈？
[Assistant]: 迴圈就像是體育課時，老師叫你繞著操場跑三圈。在程式裡，就是讓電腦重複做同一件事情

# 局限與不足

- 幻覺現象
  - GPT-3.5 有時會生成不正確或虛構的 內容，在事實性和專業知識的準確度上仍有待提升。
- 長文本處理能力有限
  - 雖然比 GPT-3 更好，但在處理極長文本或複雜推理時，仍有一定局限。
- 多模態能力缺乏
  - GPT-3.5 仍屬於單一文字模式，無法處理圖像或音訊輸入，這一點在 GPT-4 之後才有突破。

# GPT-4

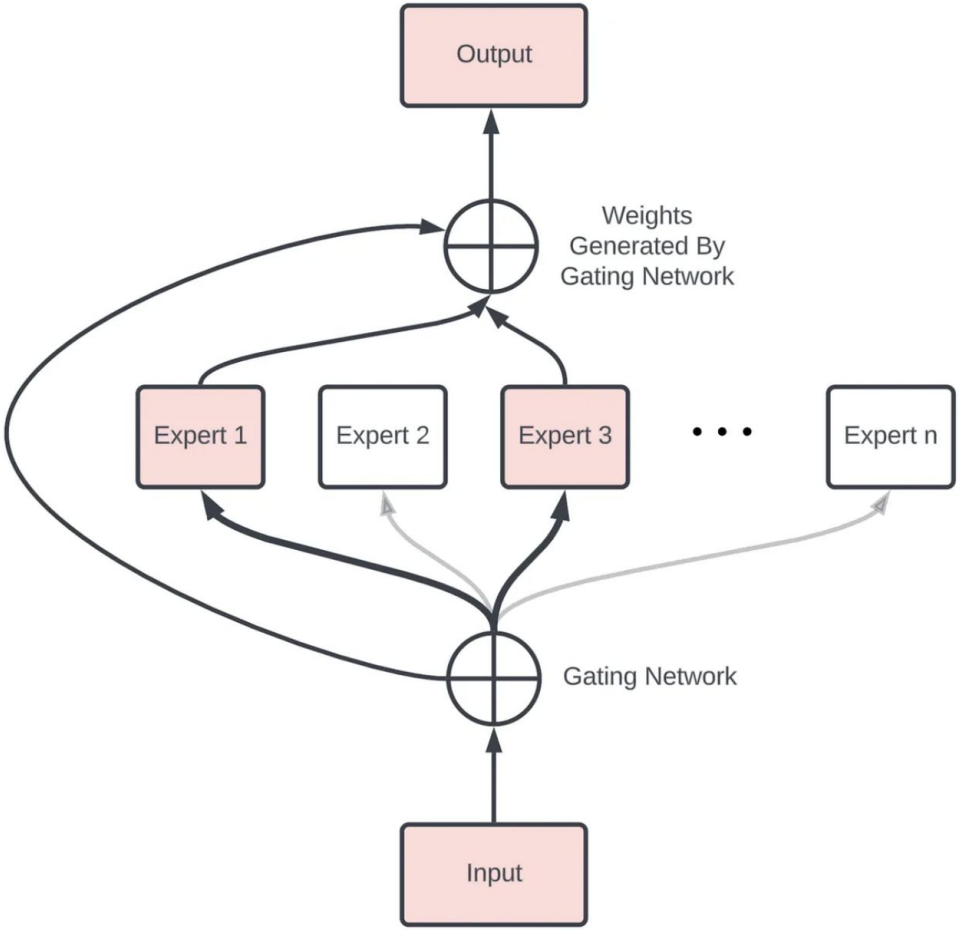
- **多模態**能力
  - GPT-4 不僅能處理文字，還能理解圖片內容，具備跨文本與圖像的分析與推理能力。
  - Ex: 進行圖像描述、設計草圖轉程式碼等任務。
- 大規模參數與強大運算力
  - GPT-4 擁有約 1.5 兆至 1.8 兆個參數（為推測值，但遠超 GPT-3 的 1,750 億），模型規模大幅提升，推理、生成能力更強。
- **更長的上下文視窗**
  - GPT-4 可處理長達 32,000 甚至 128,000 個 token 的上下文，能理解更長篇幅的對話、文件或複雜指令。
- **推理**與**解題**能力提升
  - 在標準化考試和專業領域測驗中，GPT-4 的表現接近或超越人類前10% 的水準。
- **多語言**支援
  - GPT-4 能處理多種語言，翻譯能力和語言理解大幅提升，適合跨語言溝通與應用。
- 更低的錯誤率與更高的安全性
  - 相較前代，GPT-4 生成不實或不當內容的機率顯著降低，回應更精準、相關且安全。

# GPT-4

| | GPT-3.5系列 | GPT-4系列 |
|---|---|---|
| 模態架構 | 僅支援文字的輸入與輸出 | 能同時理解文字、圖像，甚至音訊 |
| 邏輯與推理 | 適合一般問答，但在處理多步驟邏輯、數學或專業領域問題時容易產生幻覺 | 具備強大的邏輯推演能力，能精準處理程式碼、法律、醫學等高度專業的複雜指令 |
| 上下文記憶 | 記憶長度較短，長篇對話容易遺忘先前的資訊 | 支援數萬至十萬以上的Token，能一次讀取並分析 整本手冊或長篇論文 |

# Mixture of Experts (MoE)

- 傳統的模型通常是所謂的dense model，有龐大的參數量並且什麼都會
- 在訓練階段，只要模型越大，準確率就越好。
- 問題：模型越大，訓練和推論速度就越慢
- MoE的兩大核心：experts and gating network
- MoE 是一種**以空間（龐大的記憶體）換取時間（更快的運算速度）** 的優化設計




MoE diagram

Experts：

- 在模型內部，會將神經網路拆分成多個獨立的子網路，這些子網路就是**專家**。
- 每個專家在一開始的訓練過程中，會學會專精處理某種類型的任務（例如：有些專家特別擅長處理程式碼，有些專家擅長數學邏輯，有些擅長某種語言）

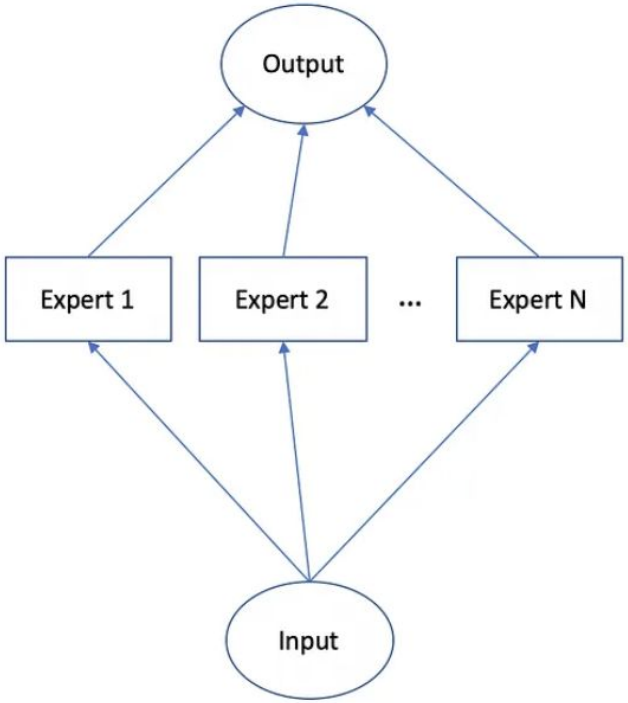
Router / Gating Network：

- 是一個輕量級的網路，負責**指派任務**
- 當我們輸入一段文字時，文字會被切成一個個Token。Gating network會針對每一個 Token 進行計算，決定這個 Token 應該交給哪幾個專家來處理

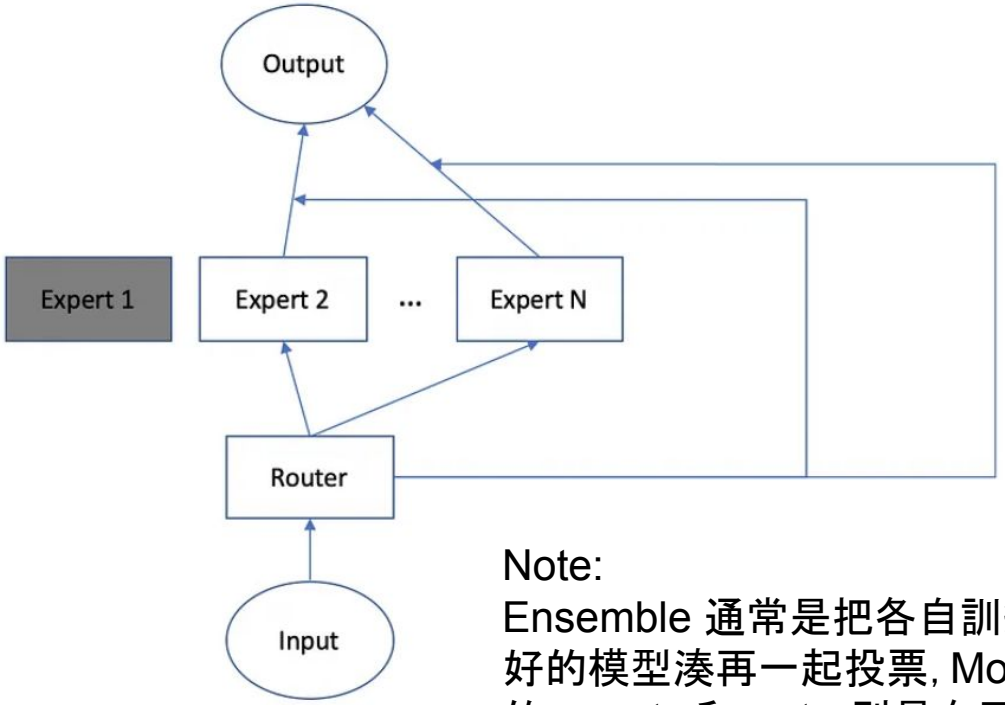
https://medium.com/@drahyhenc/mixture-of-experts%E5%AD%B8%E7%BF%92%E7%AD%86%E8%A8%98-80fae09a1b5e

https://medium.com/@drahyhenc/mixture-of-experts%E5%AD%B8%E7%BF%92%E7%AD%86%E8%A8%98-80fae09a1b5e

# MoE vs Ensemble Learning




Ensemble




MoE

Note:
Ensemble 通常是把各自訓練好的模型湊再一起投票, MoE的experts 和router則是在同一個網路裡訓練

# GPT-4o

「o」代表「Omni」，意指全能，強調其在多種資料型態（文字、語音、圖像等）上的全面處理能力。

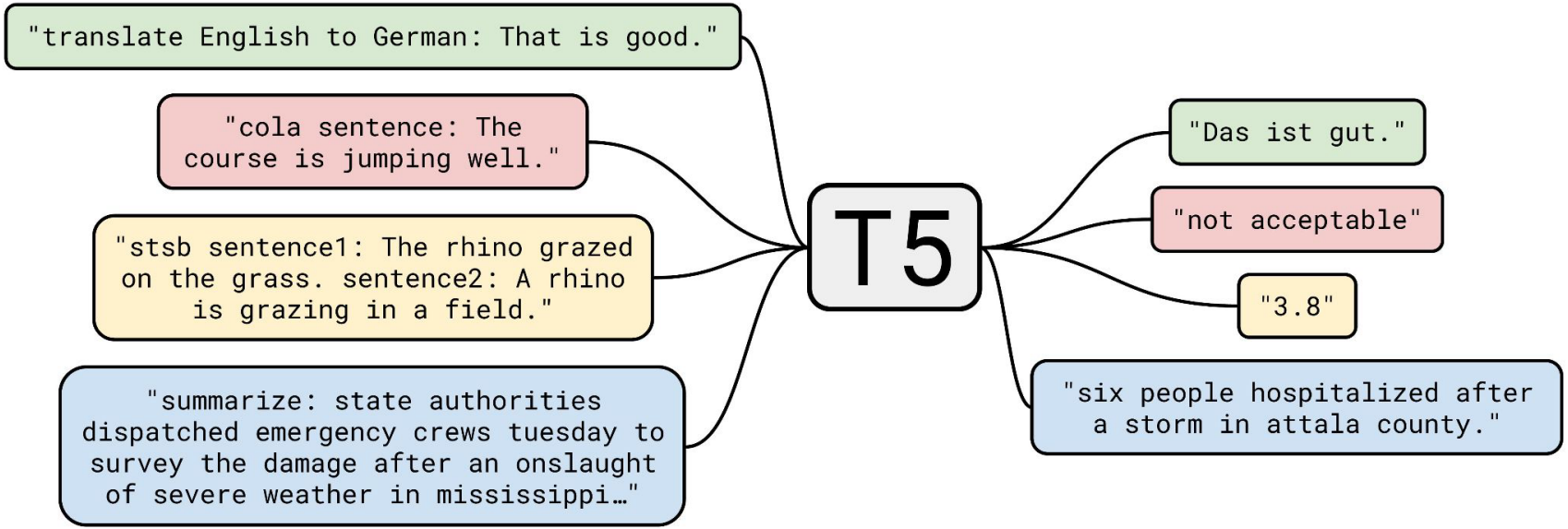
- 全模態處理
  - GPT-4 在處理語音時，要先用模型 A把語音轉文字，GPT 處理完文字後，再交給模型 B 轉回語音，過程中會遺失情緒和語氣
  - GPT-4o則是單一神經網路，直接同時接受並輸出文字、視覺、音訊，不需經過轉換
- 提供自然流暢的語音互動
  - 語音輸出自然生動，具備語調變化與 **情緒辨識** 能力，帶來近乎人類的對話體驗。
  - 支援**即時打斷**、多輪互動與**極低延遲**，讓溝通更加順暢即時。

# GPT-4o

- 實現即時多語言溝通
  - 支援超過 50 種語言，可進行無延遲的即時口譯，並自如切換不同語言，促進跨語言理解與交流。
- 支援 AI 之間的互動與協作
  - 可讓多個 AI 同時對話、協同解題，甚至共同演奏或合唱，實現創新的人機與機器協作體驗。
- 高效能、低成本的應用選擇
  - GPT-4o 的 API 回應速度為 GPT-4 的兩倍，成本降低約 50%，適用於即時、大規模的應用場 景。

# T5

- 由 Google 提出（2020）
- 設計哲學：把所有 NLP 任務都視為「文字輸入 ➜ 文字輸出」的問題
- 不論是分類、翻譯、摘要、問答…，全部任務都可以用同一個模型處理。
- 使用 Encoder-Decoder 架構（跟 Transformer 原始設計相同）
- 預訓練任務：Span Corruption（隨機刪除句中段落並要求模型重建）
- 架構中完全使用 Text 形式的 prompt，如：
  - summarize: The food was delicious and the waiter…
  - translate English to French: That is good.


"translate English to German: That is good."
"cola sentence: The course is jumping well."
"stsb sentence1: The rhino grazed on the grass. sentence2: A rhino is grazing in a field."
"summarize: state authorities dispatched emergency crews tuesday to survey the damage after an onslaught of severe weather in mississippi…"
T5
"Das ist gut."
"not acceptable"
"3.8"
"six people hospitalized after a storm in attala county."

# Span Corruption

- T5 並不是像 BERT 那樣遮蔽單字，而是遮蔽**一整段連續詞組**（span），讓模型學會生成被遮蔽的內容。
- 流程：
  - 從輸入文本中隨機遮蔽一個或多個 span
  - 每個 span 可以包含多個詞
  - 替代成特別的 placeholder（例如 <extra_id_0>、<extra_id_1>）
- 輸出序列要求模型依序生成所有被遮蔽的詞組

# Span Corruption

- 原始輸入:
  The quick brown fox jumps over the lazy dog.

- 遮蔽後輸入（Encoder Input）:
  The quick **<extra_id_0>** over the **<extra_id_1>**.

- 目標輸出（Decoder Target）:
  **<extra_id_0>** brown fox jumps **<extra_id_1>** lazy dog

# Span Corruption

- 學會根據上下文生成遺失段落，更貼近真實的 NLP 任務（如摘要、翻譯、重建）
- <extra_id_#> 是特殊的 token，T5 設計用來表示遮蔽區塊的順序
- 模型需要處理遮蔽位置與重建內容的**對齊問題**，訓練難度更高，但泛化能力也更強
  - 例如：<extra_id_0> brown fox → 下一個字還是屬於 id 0 的範圍嗎 → 是的話繼續關注 The quick 和 over the
  - 好處：可以預測不同長度的文本

# T5訓練方式

- 預訓練階段
  - 使用C4（Colossal Clean Crawled Corpus），包含大量無標註純文字
  - 做span corruption
- 微調階段：
  - 使用各式各樣有標註的標準資料集（例如 SQuAD 問答資料集、GLUE 語意測試集等）
  - 不是單一任務，而是多任務學習
  - 所有任務都轉為text-to-text格式，例如：
    - 問答：question: ... context: ... ➜ answer
    - 情感分析：sst2 sentence: I love this movie. ➜ positive
  - 可輕鬆 fine-tune 至下游任務，只需提供對應輸入與輸出

# 轉換範例

- 翻譯
  - **輸入:** translate English to French: That is good.
  - **輸出:** C’est bon.
- 摘要
  - **輸入:** summarize: The food was delicious and...
  - **輸出:** The food and service were excellent.
- 問答
  - **輸入:** question: When was NASA founded? context: ...
  - **輸出:** 1958
- 情感分析
  - **輸入:** sst2 sentence: I love this movie.
  - **輸出:** positive

# 優點與限制

優點

- 任務統一，無需為不同任務設計不同模型輸出層，易於理解與部署
- 支援多任務與遷移學習
- 高泛化能力，適合生成式任務

限制

- 對於非生成任務（如分類）效率稍低
  - 以情緒分類為例，BERT → 1, T5 →p→o→s→ i→t→i→v→e
- 對 prompt 格式較敏感，需小心設計

# BART

- Bidirectional and Auto-Regressive Transformers，由 Meta AI 提出
- 採用 Encoder-Decoder 架構
  - Encoder：類似 BERT，具備雙向注意力，可深入理解輸入
  - Decoder：類似 GPT，使用自回歸機制逐字生成輸出
- 設計目的是處理生成式 NLP 任務，如：摘要、機器翻譯、生成式問答等

# 訓練方式

- Denoising Autoencoder
  - 隨機破壞輸入句子（加 noise）
  - 訓練模型根據破壞後的輸入，還原原始句子
- 常見 noise 類型：
  - Token Masking（隱藏字詞）
  - Token Deletion（移除字詞）
  - Sentence Permutation（打亂句子順序）
  - Text Infilling（像 span corruption）

- 原始句子：
  The movie was absolutely wonderful and very emotional.
- 遮蔽後輸入：
  The movie was absolutely [MASK] and very emotional.
- 模型目標：
  學會將 [MASK] 補回為 wonderful

# 訓練方式

- Denoising Autoencoder
  - 隨機破壞輸入句子（加 noise）
  - 訓練模型根據破壞後的輸入，還原原始句子
- 常見 noise 類型：
  - Token Masking（隱藏字詞）
  - Token Deletion（移除字詞）
  - Sentence Permutation（打亂句子順序）
  - Text Infilling（像 span corruption）

- 原始句子：
  The children played happily in the park.

- 刪除後輸入：
  The children played in park.

- 模型目標：
  還原為 The children played happily in the park.

# 訓練方式

- Denoising Autoencoder
  - 隨機破壞輸入句子（加 noise）
  - 訓練模型根據破壞後的輸入，還原原始句子
- 常見 noise 類型：
  - Token Masking（隱藏字詞）
  - Token Deletion（移除字詞）
  - Sentence Permutation（打亂句子順序）
  - Text Infilling（像 span corruption）

- 原始段落：
  1. He went to the store.
  2. Then he bought some groceries.
  3. After that, he returned home.

- 打亂後輸入：
  After that, he returned home. He went to the store. Then he bought some groceries.

- 模型目標：
  重新排列為原本的順序。

# 訓練方式

- Denoising Autoencoder
  - 隨機破壞輸入句子（加 noise）
  - 訓練模型根據破壞後的輸入，還原原始句子
- 常見 noise 類型：
  - Token Masking（隱藏字詞）
  - Token Deletion（移除字詞）
  - Sentence Permutation（打亂句子順序）
  - Text Infilling（像 span corruption）

- 原始句子：
  The cat sat on the warm, sunny windowsill all afternoon.

- 刪除區塊：
  The cat <mask> all afternoon.

- 模型目標：
  補回為 sat on the warm, sunny windowsill

# BART 的應用任務

Note:
針對分類的任務的輸出，BART的做法比較類似RNN，而不是T5

- 翻譯
  - **輸入：** That is good.
  - **輸出：** C’est bon.
- 摘要
  - **輸入：** The food was delicious and...
  - **輸出：** The food and service were excellent.
- 問答
  - **輸入：** **[question]** When was NASA founded? **[context]** ... (optional)
  - **輸出：** 1958
- 情緒分析
  - **輸入：** I love this movie.
  - **輸出：** positive

# BART vs T5

| 任務 | BART | T5 |
| --- | --- | --- |
| 預訓練目標 | denoising autoencoder | span corruption |
| 模型框架 | Encoder-Decoder | Encoder-Decoder |
| 輸入格式 | 任務需自行設計格式 | 統一為 text-to-text，使用 prompt |
| 任務泛化能力 | 高，但不如 T5 直觀 | 更統一與靈活 |